

车辆维修管理系统 - 存储过程与触发器说明

生成日期：2025-01-12

数据库：MySQL 8.0+

ORM框架：Sequelize (Node.js)

1. 系统架构设计说明

1.1 当前设计选择

车辆维修管理系统采用了现代化的架构设计，选择在**应用层**而非数据库层实现业务逻辑。具体设计特点：

- ORM框架**：使用Sequelize进行数据库操作
- 业务逻辑**：在Node.js应用层实现
- 数据一致性**：通过事务和应用层验证保证
- 触发器替代**：使用Sequelize的hooks机制

1.2 设计优势

- 跨数据库兼容性**：业务逻辑不依赖特定数据库的存储过程语法
- 维护便利性**：业务逻辑与应用代码统一管理
- 测试友好性**：可以对业务逻辑进行单元测试
- 版本控制**：业务逻辑变更可以通过Git进行版本管理
- 调试便利**：可以使用Node.js调试工具

2. 当前系统中的应用层实现

2.1 Sequelize Hooks (替代触发器)

系统使用Sequelize的hooks机制来实现原本需要触发器的功能：

2.1.1 支付金额自动计算

```
// 在 Payment 模型中
Payment.define('Payment', {
  // ... 字段定义
}, {
  hooks: {
    beforeSave: (payment, options) => {
      // 自动计算总费用
      payment.total_fee = (parseFloat(payment.labor_fee) || 0) +
        (parseFloat(payment.material_fee) || 0);
    }
  }
});
```

2.1.2 库存扣减验证

```
// 在 WorkOrderMaterial 模型中
WorkOrderMaterial.addMaterial = async function(orderId, partId, quantity, price) {
  // 事务开始
  const transaction = await sequelize.transaction();

  try {
    // 检查库存
    const currentStock = await InventoryTxn.getCurrentStock(partId);
    if (currentStock < quantity) {
      throw new Error(`库存不足, 当前库存: ${currentStock}, 需要: ${quantity}`);
    }

    // 创建工单配件记录
    const workOrderMaterial = await WorkOrderMaterial.create({
      order_id: orderId,
      part_id: partId,
      qty: quantity,
      price: price
    }, { transaction });

    // 创建库存出库记录
    await InventoryTxn.recordOutbound(partId, quantity, orderId, { transaction });

    // 提交事务
    await transaction.commit();
    return workOrderMaterial;
  } catch (error) {
    await transaction.rollback();
    throw error;
  }
};
```

2.2 业务逻辑方法 (替代存储过程)

2.2.1 工单状态流转

```
// 在 WorkOrder 模型中
WorkOrder.prototype.updateStatus = async function(newStatus, reason = null) {
  // 验证状态流转的合法性
  const validTransitions = {
    'pending': ['assigned', 'cancel'],
    'assigned': ['in_progress', 'cancel'],
    'in_progress': ['done', 'cancel'],
    'done': [],
    'cancel': []
  };

  if (!validTransitions[this.status].includes(newStatus)) {
```

```
    throw new Error(`无法从状态 ${this.status} 转换到 ${newStatus}`);
  }

  this.status = newStatus;
  if (newStatus === 'done') {
    this.finished_at = new Date();
  }

  return await this.save();
};
```

2.2.2 费用计算

```
// 在 WorkOrder 模型中
WorkOrder.prototype.calculateTotalCost = async function() {
  // 计算配件费用
  const materialCost = await WorkOrderMaterial.findAll({
    where: { order_id: this.order_id },
    attributes: [
      [sequelize.fn('SUM', sequelize.literal('qty * price')),
      'total_material_cost']
    ],
    raw: true
  });

  // 计算人工费用
  const laborCost = await WorkOrderMechanic.findAll({
    where: { order_id: this.order_id },
    include: [{
      model: MechanicProfile,
      as: 'mechanic',
      attributes: ['hourly_rate']
    }],
    attributes: [
      [sequelize.fn('SUM', sequelize.literal('hours_worked *
mechanic.hourly_rate')), 'total_labor_cost']
    ],
    raw: true
  });

  return {
    material_cost: parseFloat(materialCost[0].total_material_cost) || 0,
    labor_cost: parseFloat(laborCost[0].total_labor_cost) || 0
  };
};
```

3. 传统存储过程与触发器的实现建议

尽管系统当前使用应用层实现，但如果需要改为数据库层实现，以下是建议的存储过程和触发器设计：

3.1 存储过程设计

3.1.1 创建工单存储过程

```
DELIMITER //
```

```
CREATE PROCEDURE CreateWorkOrder(  
    IN p_vehicle_id BIGINT,  
    IN p_customer_id BIGINT,  
    IN p_description TEXT,  
    IN p_trade ENUM('engine','paint','electric'),  
    OUT p_order_id BIGINT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    START TRANSACTION;  
  
    -- 插入工单  
    INSERT INTO work_orders (  
        vehicle_id,  
        customer_id,  
        status,  
        description,  
        trade,  
        created_at  
    ) VALUES (  
        p_vehicle_id,  
        p_customer_id,  
        'pending',  
        p_description,  
        p_trade,  
        NOW()  
    );  
  
    SET p_order_id = LAST_INSERT_ID();  
  
    -- 创建支付记录  
    INSERT INTO payments (order_id, labor_fee, material_fee, total_fee)  
    VALUES (p_order_id, 0, 0, 0);  
  
    COMMIT;  
END //
```

```
DELIMITER ;
```

3.1.2 工单费用计算存储过程

```
DELIMITER //
```

```
CREATE PROCEDURE CalculateOrderCost(  
    IN p_order_id BIGINT,  
    OUT p_labor_fee DECIMAL(10,2),  
    OUT p_material_fee DECIMAL(10,2),  
    OUT p_total_fee DECIMAL(10,2)  
)  
BEGIN  
    -- 计算人工费  
    SELECT COALESCE(SUM(wom.hours_worked * mp.hourly_rate), 0)  
    INTO p_labor_fee  
    FROM work_order_mechanics wom  
    INNER JOIN mechanic_profiles mp ON wom.mechanic_id = mp.mechanic_id  
    WHERE wom.order_id = p_order_id;  
  
    -- 计算配件费  
    SELECT COALESCE(SUM(wom.qty * wom.price), 0)  
    INTO p_material_fee  
    FROM work_order_materials wom  
    WHERE wom.order_id = p_order_id;  
  
    -- 计算总费用  
    SET p_total_fee = p_labor_fee + p_material_fee;  
  
    -- 更新支付记录  
    UPDATE payments  
    SET labor_fee = p_labor_fee,  
        material_fee = p_material_fee,  
        total_fee = p_total_fee  
    WHERE order_id = p_order_id;  
END //
```

```
DELIMITER ;
```

3.1.3 库存管理存储过程

```
DELIMITER //
```

```
CREATE PROCEDURE ProcessInventoryTransaction(  
    IN p_part_id BIGINT,  
    IN p_order_id BIGINT,  
    IN p_qty INTEGER,  
    IN p_type ENUM('IN', 'OUT', 'ADJUST')  
)  
BEGIN  
    DECLARE v_current_stock INTEGER DEFAULT 0;  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;
```

```
RESIGNAL;  
END;  
  
START TRANSACTION;  
  
-- 获取当前库存 (加锁)  
SELECT qty INTO v_current_stock  
FROM parts  
WHERE part_id = p_part_id  
FOR UPDATE;  
  
-- 检查出库操作的库存充足性  
IF p_type = 'OUT' AND v_current_stock < ABS(p_qty) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = '库存不足, 无法完成出库操作';  
END IF;  
  
-- 插入库存交易记录  
INSERT INTO inventory_txns (part_id, order_id, qty, type, created_at)  
VALUES (p_part_id, p_order_id, p_qty, p_type, NOW());  
  
-- 更新配件库存  
UPDATE parts  
SET qty = qty + p_qty  
WHERE part_id = p_part_id;  
  
COMMIT;  
END //  
  
DELIMITER ;
```

3.2 触发器设计

3.2.1 支付金额自动计算触发器

```
DELIMITER //  
  
CREATE TRIGGER tr_payments_before_insert  
BEFORE INSERT ON payments  
FOR EACH ROW  
BEGIN  
    SET NEW.total_fee = COALESCE(NEW.labor_fee, 0) + COALESCE(NEW.material_fee,  
0);  
END //  
  
CREATE TRIGGER tr_payments_before_update  
BEFORE UPDATE ON payments  
FOR EACH ROW  
BEGIN  
    SET NEW.total_fee = COALESCE(NEW.labor_fee, 0) + COALESCE(NEW.material_fee,  
0);
```

```
END //

DELIMITER ;
```

3.2.2 工单状态变更日志触发器

```
-- 首先创建状态变更日志表
CREATE TABLE work_order_status_log (
    log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    old_status ENUM('pending','assigned','in_progress','done','cancel'),
    new_status ENUM('pending','assigned','in_progress','done','cancel'),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES work_orders(order_id) ON DELETE CASCADE
);

DELIMITER //

CREATE TRIGGER tr_work_orders_status_change
AFTER UPDATE ON work_orders
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO work_order_status_log (order_id, old_status, new_status,
            changed_at)
            VALUES (NEW.order_id, OLD.status, NEW.status, NOW());

        -- 如果状态变为完成, 自动设置完成时间
        IF NEW.status = 'done' AND OLD.status != 'done' THEN
            UPDATE work_orders
            SET finished_at = NOW()
            WHERE order_id = NEW.order_id;
        END IF;
    END IF;
END //

DELIMITER ;
```

3.2.3 库存变动触发器

```
DELIMITER //

CREATE TRIGGER tr_inventory_txns_after_insert
AFTER INSERT ON inventory_txns
FOR EACH ROW
BEGIN
    -- 更新配件表中的库存数量
    UPDATE parts
```

```
SET qty = qty + NEW.qty
WHERE part_id = NEW.part_id;

-- 如果是出库操作且库存不足，回滚操作
IF NEW.type = 'OUT' AND (SELECT qty FROM parts WHERE part_id = NEW.part_id) <
0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = '库存不足，无法完成出库操作';
END IF;
END //
```

DELIMITER ;

4. 性能对比分析

4.1 应用层实现 vs 数据库层实现

方面	应用层实现	数据库层实现
性能	网络往返较多	数据库内部执行，性能更优
可维护性	代码统一管理，便于维护	需要同时维护应用代码和数据库代码
可测试性	易于单元测试	需要专门的数据库测试工具
跨平台性	数据库无关	依赖特定数据库语法
调试便利性	应用层调试工具丰富	数据库调试工具有限
版本控制	代码版本控制	需要额外的数据库版本管理

4.2 推荐使用场景

4.2.1 推荐使用存储过程的场景

- 复杂的数据统计和报表生成
- 大批量数据处理操作
- 对性能要求极高的场景
- 数据库管理员主导的项目

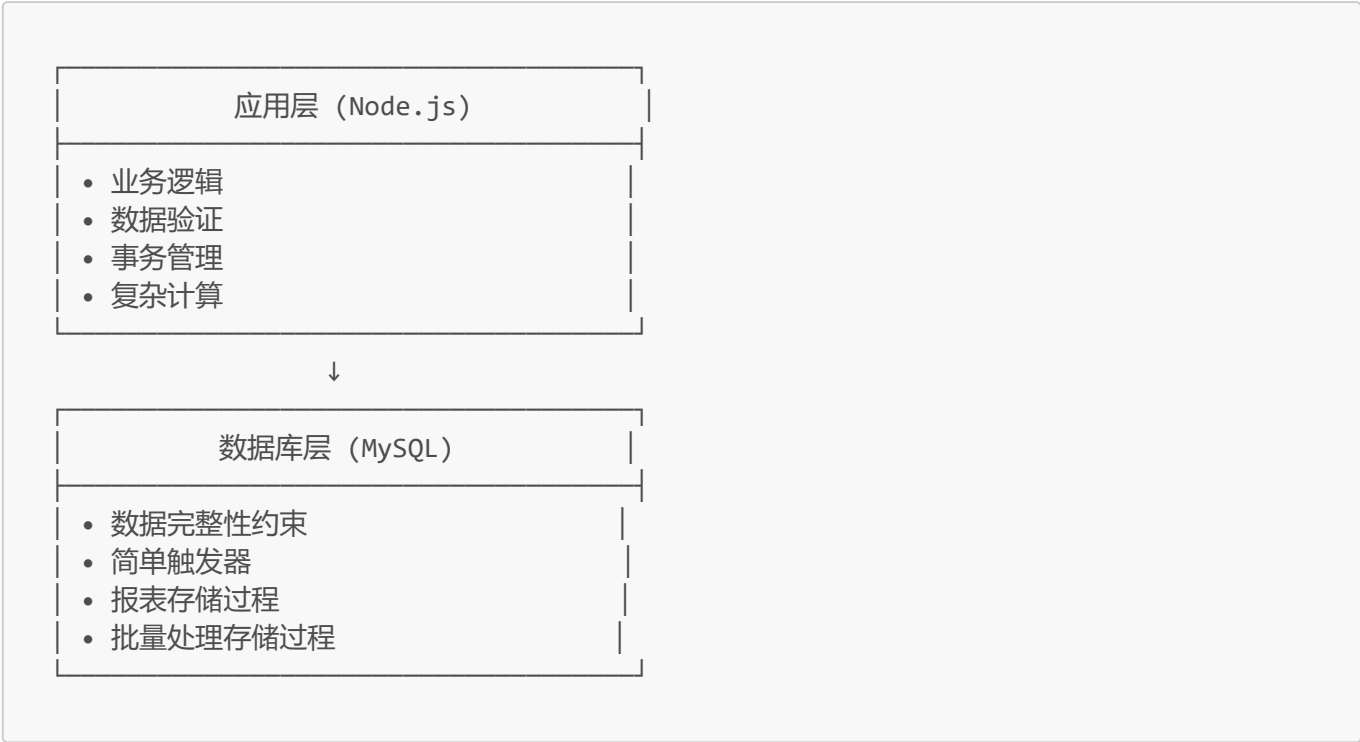
4.2.2 推荐使用应用层实现的场景

- 业务逻辑复杂且变化频繁
- 需要跨数据库兼容性
- 团队以应用开发为主
- 重视代码的可测试性和可维护性

5. 混合架构建议

5.1 分层设计原则

对于企业级应用，建议采用混合架构：



5.2 具体实现建议

5.2.1 应用层负责

- 业务逻辑验证
- 工单状态流转
- 用户权限检查
- 复杂计算逻辑

5.2.2 数据库层负责

- 数据完整性约束
- 简单的数据计算触发器
- 批量数据处理存储过程
- 系统维护存储过程

6. 实施建议

6.1 当前系统优化

如果要在当前系统中增加存储过程和触发器，建议按以下步骤进行：

1. **评估需求**：确定哪些业务逻辑适合移到数据库层
2. **渐进实施**：先实施非关键功能，验证效果
3. **性能测试**：对比应用层和数据库层实现的性能差异
4. **文档完善**：建立数据库对象的文档和版本管理

6.2 监控和维护

1. **性能监控**：监控存储过程的执行时间和资源使用
 2. **日志记录**：记录存储过程的执行日志
 3. **版本管理**：建立数据库对象的版本控制流程
 4. **定期审计**：定期检查存储过程和触发器的有效性
-

7. 总结

车辆维修管理系统当前采用应用层实现业务逻辑的架构，这种设计选择在当前阶段是合适的。如果未来需要优化性能或处理大规模数据，可以考虑将部分功能迁移到数据库层使用存储过程和触发器实现。

关键建议：

- 保持当前架构的灵活性
- 在性能瓶颈出现时，有选择地使用存储过程
- 建立完善的监控和维护体系
- 优先考虑代码的可维护性和可测试性